MINOR PROJECT · 2026–27

JASIPA

Job Applicant Screening and Interview Panel Agent

[Team member names]
Guide: [Guide name]
Dept. of CSE (Artificial Intelligence)

An HR agent that

1. **Screens resumes** against a versioned, job-anchored rubric
2. **Assesses skills** with generated questions and sandboxed code
3. **Convenes a panel** of three independent agents
4. **Checks for bias** on every merged decision
5. **Leaves the decision** to a human, with a verifiable audit trail

PROBLEM

Screening is slow, inconsistent and hard to audit

Recruitment AI is classed as high-risk under the EU AI Act; NYC Local Law 144 requires bias audits.

Inconsistent evaluation

Different reviewers weigh the same resume differently; criteria drift between rounds and roles.

Unchecked disparity

Small applicant pools make adverse-impact checks easy to misread: no data is often reported as no bias.

Opaque automation

Automated rejection without human review or an audit trail is unacceptable for hiring decisions.

Problem statement

Build an HR agent that takes a candidate from resume screening through skill assessment to an interview-panel recommendation, using resume_screen, skill_assess and bias_check tools, a JD-anchored rubric, stage memory, an MCP connector to an ATS database and a swarm of evaluator agents — where no candidate is rejected without human review and every panel decision carries a logged bias check.

OBJECTIVES

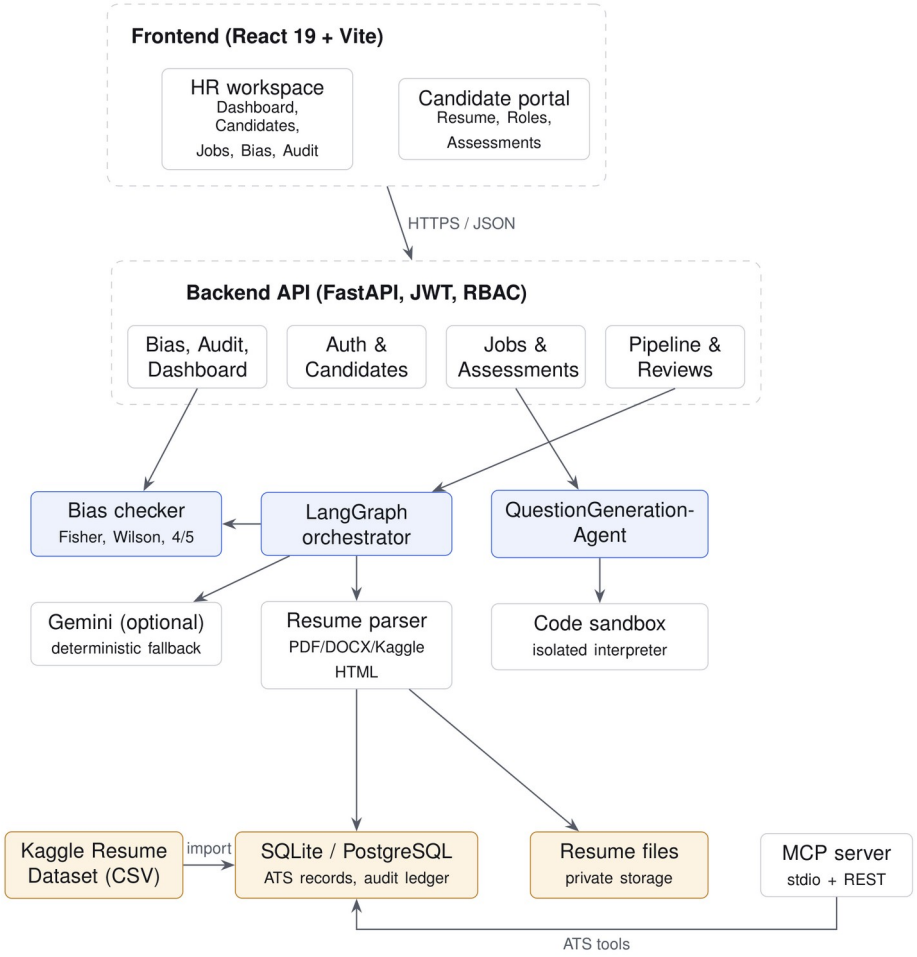
Objectives and status

#	Objective	Status
O1	Screen PDF, DOCX and dataset resumes against a versioned rubric	**Done**
O2	Generate MCQ, coding, scenario and STAR assessments; regenerate	**Done**
O3	Score code against hidden tests in an isolated sandbox	**Done**
O4	Swarm of three evaluators + Panel Coordinator (LangGraph)	**Done**
O5	Bias check on every panel decision, small-sample aware	**Done · no real cohort data**
O6	Human decisions, two-reviewer escalation, auditable log	**Done**
O7	MCP connector to the ATS database	**Done**
O8	Kaggle Resume Dataset as candidate data	**Done · 6 categories**
O9	Agents that learn from decisions and outcomes	**Not implemented**

DESIGN

System architecture

React SPA → FastAPI → agents, sandbox and bias services → SQL database; MCP connector for agents.

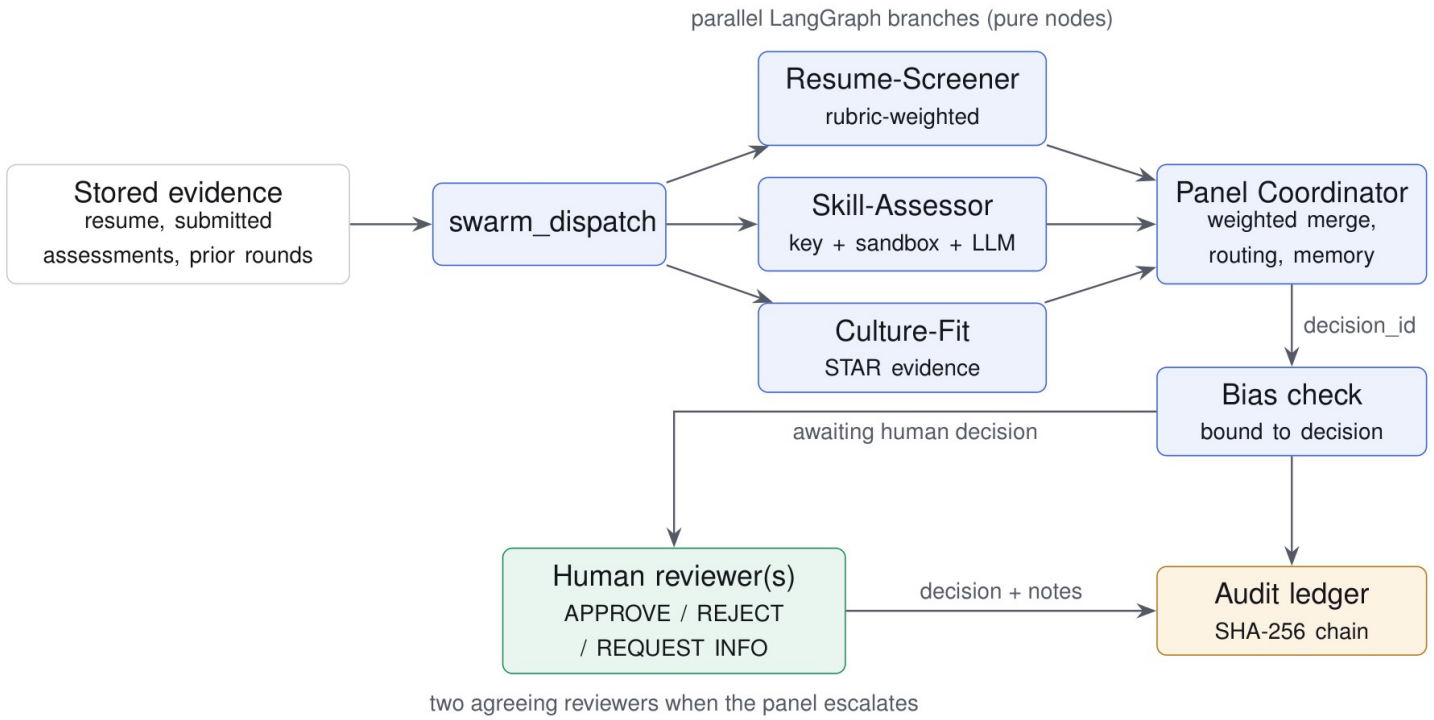


- **Frontend:** React 19 + Vite; HR workspace and candidate portal; light, minimal design system.
- **API:** FastAPI with JWT, bcrypt and role-based access (HR / candidate).
- **Agents:** LangGraph state graph; each agent is a chain of strategies — LLM → rules → human expert.
- **Services:** resume parser, question generator, code sandbox, fairness statistics, audit chain.
- **Data:** SQLAlchemy over SQLite / PostgreSQL; versioned rubrics; candidate stage memory.
- **Connector:** MCP stdio JSON-RPC + REST facade over ATS records — no decision tool exposed.

ORCHESTRATION

Agentic workflow

Three evaluators run in parallel on separate evidence and never see each other's output.



- **Resume-Screener** — resume vs. rubric (redacted, injection-screened)
- **Skill-Assessor** — answer keys, sandbox pass rate, open answers
- **Culture-Fit** — STAR evidence per competency
- **Panel Coordinator** — weighted merge (0.4 / 0.4 / 0.2), routes only
- **Bias check** — bound to the panel decision
- **Human reviewer** — records the decision; 2 reviewers if agents diverge ≥ 30 pts

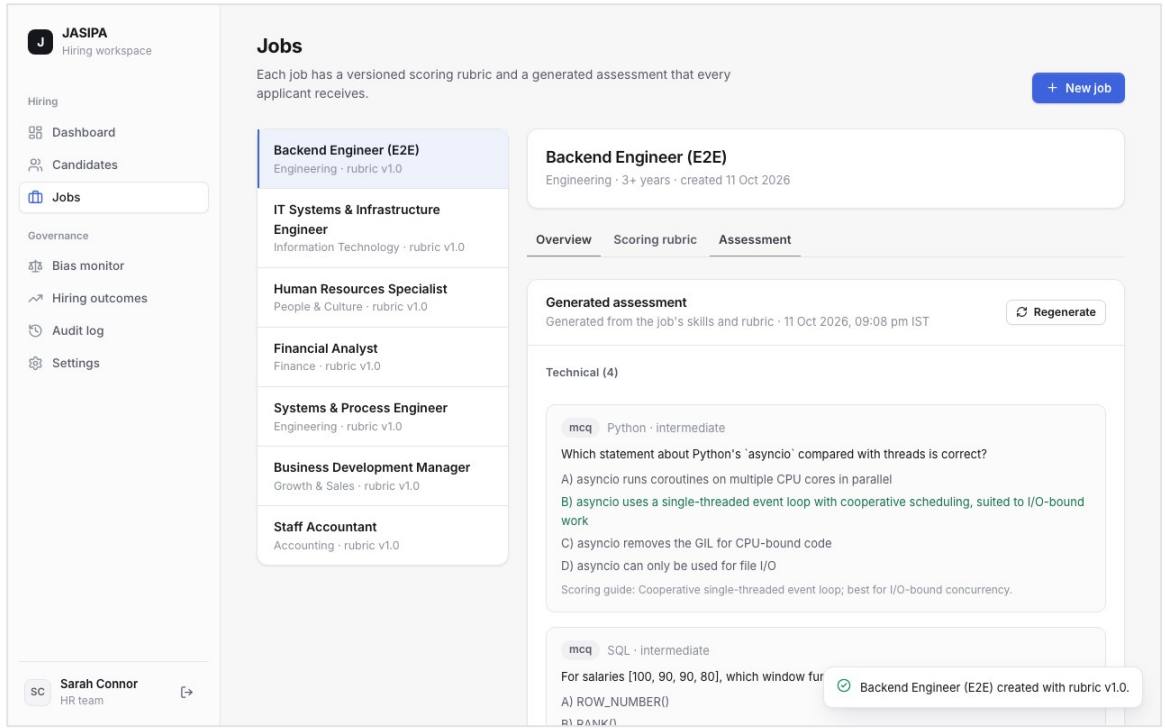
IMPLEMENTATION

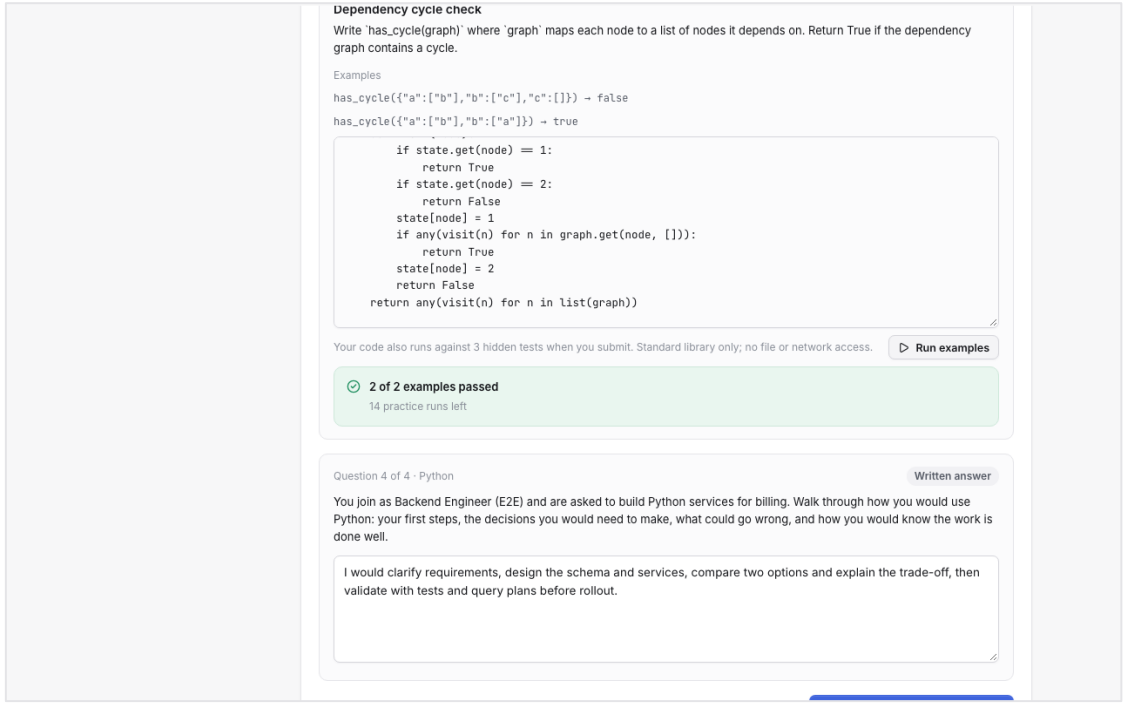
Agentic AI components

Component	Implementation in JASIPA
Tools	resume_screen · skill_assess (with code sandbox) · bias_check · MCP ATS tools
Skill	Versioned JD-anchored rubric: resume 50/15/20/15, technical 30/40/30, competencies = 100
Memory	Candidate stage machine, panel history across rounds, cached question sets, audit history
Connector	MCP (initialize, tools/list, tools/call) over stdio, plus REST facade
Orchestration	LangGraph StateGraph: dispatch → 3 parallel evaluators → panel → bias
Model	Optional Gemini 2.5 Flash, JSON output, temperature 0.2 — deterministic fallbacks offline
Governance	Routing-only labels · human decision endpoint · two-reviewer rule · hash-chained log

QUESTIONGENERATIONAGENT

Role-specific assessments and sandboxed code





Generation MCQ, coding (programming roles only), scenario and STAR questions from the JD and rubric; cached per rubric version. Regenerate saves only a set that differs, and reports how many questions changed.

Sandbox Fresh isolated interpreter, CPU / memory / file / process limits, audit hook blocks network, files and subprocesses. Judging happens outside the sandbox, so forged output cannot pass.

FAIRNESS

Bias check: three honest states

Cohort = HR-recorded monitoring label; never inferred from resumes, never shown to agents.

Disparity detected

Impact ratio < 0.80 (four-fifths screen) with n ≥ 5 in each cohort. Fisher p shown as significant / not significant.

No indicator detected

Screen not triggered. A significant rate difference alone is a note, not a warning. Under n = 30: low-power caution.

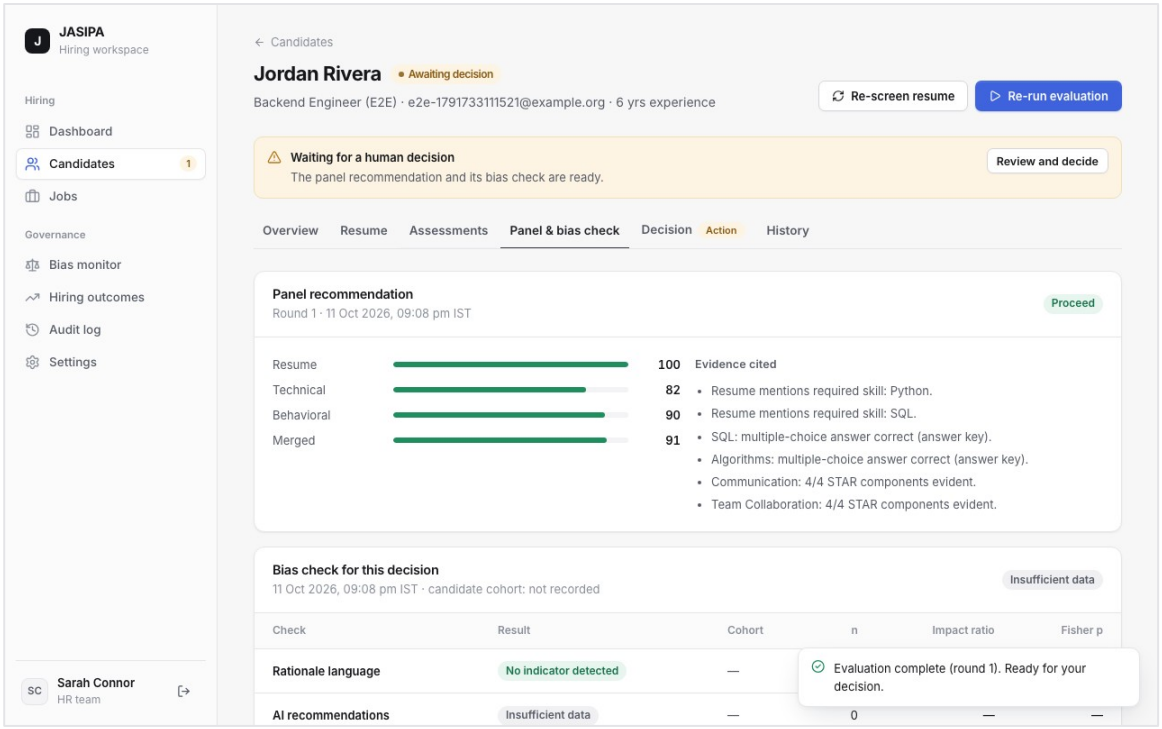
Insufficient data

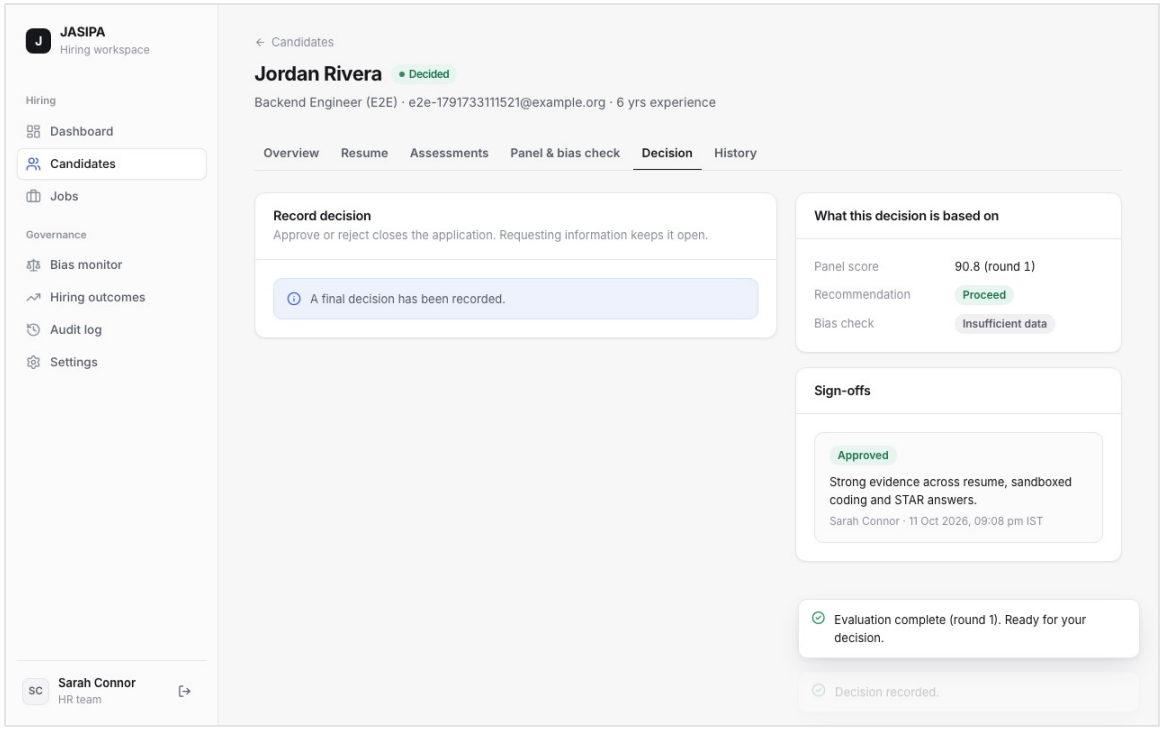
Cohort below 5, fewer than two eligible cohorts, or no selections. Never presented as fairness.

- **Per decision:** AI recommendation, final human decision, post-hire outcome, score-threshold gap and rationale language.
- **Uncertainty:** Wilson 95% intervals; fragility flag when one outcome would flip the screen; Fisher test verified against SciPy.
- **Separation:** the bias check never changes scores or routing — it informs the human reviewer.

HUMAN OVERSIGHT

Panel, bias check and human decision





- A decision is accepted only for the latest panel decision with a bias check, and binds both IDs to the review.
- Every action is appended to a SHA-256 hash chain; Audit log → Verify integrity recomputes it.

DATASET

Data: Kaggle Resume Dataset

Seeded synthetic candidates were replaced by real resumes; nothing is invented.

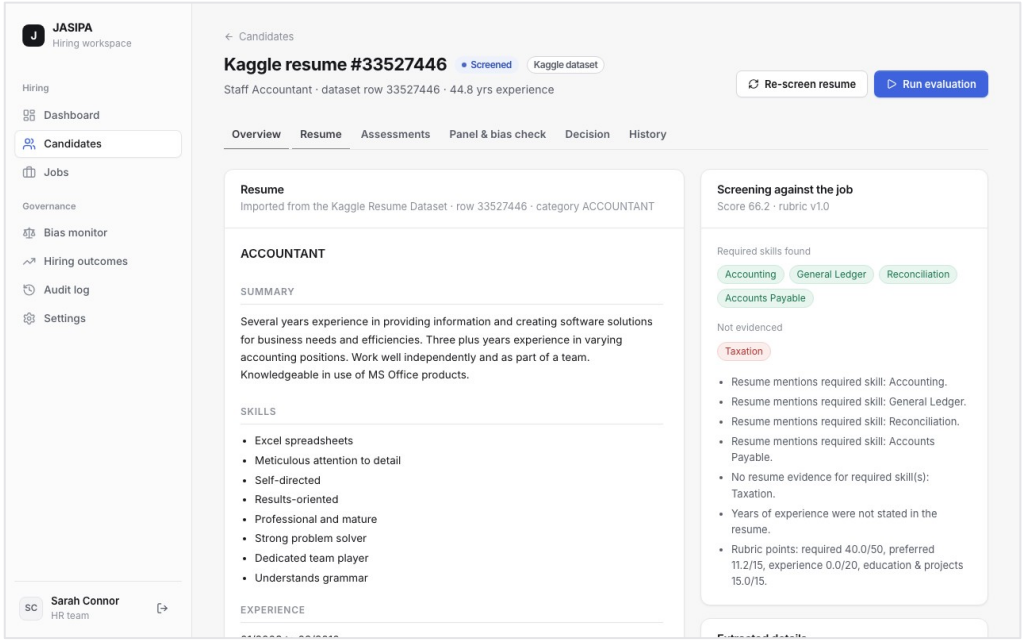
2,484
resumes in Resume.csv

24
job categories

6
categories with authored JDs

48
records imported by default

- **Categories:** IT, Engineering, HR, Finance, Accountant, Business Development.
- **Structure:** sections rebuilt from Resume_html, so the resume reads like the original.
- **Provenance:** source = dataset_import, external ID, category; badge in the UI; separate from real applicants.
- **Honest gaps:** no names, cohorts, answers or outcomes — so dataset runs report incomplete evidence and insufficient data.



VALIDATION

Testing: 149 / 149 passed

Generated from the runners' result files by scripts/run_all_tests.py — not written by hand.

Suite	Total	Pass	Fail
Backend (pytest)	110	110	0
Frontend (Vitest)	30	30	0
End-to-end (Playwright)	9	9	0
All	**149**	**149**	**0**

- **Unit:** agents, sandbox escapes, question generation, Fisher / Wilson values, resume parsing.
- **API:** RBAC, uploads, regeneration, decisions, two reviewers, audit chain, MCP.
- **Component:** IST formatting, bias labels, resume rendering, regeneration feedback.
- **End-to-end:** 9 Playwright journeys — job → application → sandboxed code → panel → bias → decision → audit.
- **Defects found:** 'C' matched inside 'C++'; regeneration reported false success; error-check ordering; sticky sidebar.

HONEST ASSESSMENT

Does the agent learn? No — by design

Mechanism	Present	What it does
Rubric / rule-based scoring	**Yes**	Default for every agent; deterministic and inspectable
LLM inference	**Optional**	Scoring open answers, drafting questions, summaries — cannot override objective scores
Stored history	**Yes**	Screenings, assessments, panels, bias checks, reviews, outcomes, audit
Round memory	**Context only**	Previous panel rounds are reported to the coordinator; scores unchanged
Retrieval	**No**	No vector store or case retrieval
Training / feedback loop	**No**	Hiring outcomes are descriptive analytics; rubric changes are manual

Why: learning from historical hiring outcomes can encode historical bias. Any feedback loop needs a validated, audited, human-approved process — stored outcomes are the foundation for it.

CONCLUSION

Limitations and future scope

Limitations

- No learning loop from decisions or outcomes
- No real cohort data — bias paths validated on controlled tests
- Dataset has no outcomes, so accuracy vs. hiring ground truth is not measured
- LLM paths tested with mocks; output quality not evaluated
- Subprocess sandbox is defence in depth; Docker recommended
- Low statistical power under 30 candidates per cohort

Future scope

- Audited, human-approved rubric updates from outcomes
- Retrieval over past evaluations for reviewer consistency
- Agreement metrics against expert ratings on a labelled sample
- More dataset categories and sandbox languages
- Container / VM sandbox by default; production ATS over MCP
- Intersectional cohorts and power analysis

Thank you

Questions and live demo

github.com/UditaB20/jasipa